SLE-4: Final Viva & Architecture Decision Record (ADR)

Course: 02AML204 — Introduction to Artificial Intelligence

PRN: 25UAM105
Name: Pranav Mane
Division: B

PART A: Architecture Decision Record (ADR)

Title: Use DFS (stack-based) as the default solver for the perfect maze, keeping BFS behind the same interface as a baseline and fallback

Status: Accepted

1. Context

The system generates a 15×15 perfect maze (225 cells) and must solve it from (0,0) to (14,14). In SLE-1 I implemented two uninformed search agents, BFS and DFS, and in SLE-2 I profiled both on the same fixed maze (50 runs each, perf_counter cross-checked with cProfile). In the SLE-3 C4 diagrams the Search Engine container is separate from the Maze Store and the Profiling Harness, and only the Frontier component differs between the two algorithms. I therefore had to decide which algorithm the system should use by default.

2. Decision

I decided to use DFS, with a stack as its Frontier, as the default solver for this system. BFS stays in the code behind the same bfs()/dfs() interface, so it can be swapped in as the baseline for comparison and as the fallback whenever a maze can contain loops or multiple routes.

3. Alternatives Considered

- **Alternative 1: BFS (queue) as the default.** Guarantees the shortest path, but in a perfect maze that guarantee is not needed.
- **Alternative 2: A* with a Manhattan-distance heuristic.** Would add a Heuristic Module and a priority queue; not justified for a 225-cell maze with a single path.
- **Alternative 3: Greedy Best-First Search.** Also needs a heuristic, gives no optimality guarantee, and can be misled by walls in a maze.

4. Rationale / Justification

The decision rests on my SLE-2 measurements and my SLE-3 design:

Metric (SLE-2, 15×15 maze)	BFS	DFS	DFS advantage
Avg. time (ms), 50 runs	0.1142	0.0946	≈17% lower
Worst time (ms)	0.1846	0.1160	≈37% lower
Nodes expanded	212	174	38 fewer (≈18%)
Path length (moves)	122	122	tie

- **Same answer, lower cost.** Both algorithms returned the identical 122-move path, because a perfect maze has exactly one simple path between any two cells. So BFS's shortest-path guarantee gives no benefit here, yet its level-by-level expansion still costs extra work.
- **Measured advantage.** DFS expanded 174 nodes against 212 for BFS (about 1.22× fewer) and averaged 0.0946 ms against 0.1142 ms (about 1.21× faster). It was also more consistent: the worst run was 0.1160 ms for DFS against 0.1846 ms for BFS.
- **Architecture makes it low-risk.** From the SLE-3 component diagram, only the Frontier differs between the algorithms. Choosing DFS as default therefore does not lock the system in; BFS can be switched back on without changing the Maze Store, Visited Set or Profiling Harness.

- **Simpler than A* / Greedy.** With no heuristic needed, the design stays at six containers, and there is no extra priority-queue or heuristic code to test.

5. Consequences

Positive:

- Fewer nodes expanded and lower average and worst-case time on the profiled maze.
- Stack-based frontier is simple to implement and uses little memory on a single-corridor maze.
- Algorithms remain interchangeable, so future comparison (including A*) is easy.

Negative / Trade-offs:

- DFS does not guarantee the shortest path. If the maze is changed to allow loops or several routes, DFS may return a much longer path, and BFS must become the default again.
- The speed gain is small in absolute terms (about 0.02 ms on average) because the maze is tiny, and node count was taken from one representative run per algorithm.
- Results come from one fixed maze, so they should not be generalised to every maze without re-profiling.

6. References (optional)

SLE-2 Profiling Report (results table, Section 3); SLE-3 C4 Model (Container and Component diagrams).

PART B: Short Summary of My SLE Journey

In SLE-1 I built a maze generator with BFS and DFS agents and an AI contribution log. In SLE-2 I profiled both on the same 15×15 maze, 50 runs each, and found that DFS expanded fewer nodes (174 vs 212) and was faster on average (0.0946 ms vs 0.1142 ms), with both returning the same 122-move path. In SLE-3 I documented the system with C4 diagrams: six containers, and a Search Engine whose only varying part is the Frontier. This ADR joins the three: the SLE-2 numbers justify choosing DFS, and the SLE-3 structure keeps BFS available if the problem changes.

PART C: AI Contribution Note

- **AI tools used:** Claude (Anthropic).
- **What AI helped with:** code structure for the maze, BFS/DFS and profiling loop (SLE-1/2); drafts of the C4 diagrams and report text (SLE-3); and a first draft of this ADR and the viva notes, based on my own SLE-2 and SLE-3 work.
- **What I did myself:** I ran the SLE-2 code on my own machine and verified the timing numbers. I reviewed the code and checked that the names and structure in the C4 diagrams match my program. _____ (add anything else that is true for SLE-4, e.g. “I practised explaining the ADR aloud and checked every number against my SLE-2 table.”)

PART D: Viva Preparation Notes

- **Why DFS?** Perfect maze → only one path, so BFS's shortest-path guarantee is unused; DFS expanded 174 vs 212 nodes and averaged 0.0946 vs 0.1142 ms.
- **Why not A*/Greedy?** No heuristic needed at this size; would add a Heuristic Module and priority queue with no clear gain.
- **What if the maze gets bigger?** BFS's queue grows with the size of each distance “ring”; DFS depends on dead ends along the way. DFS may stay ahead on a perfect maze, but I would re-profile.
- **What if the maze has loops?** DFS may return a longer path; switch the default back to BFS (same interface, only the Frontier changes).
- **One-minute C4 answer:** Student → system → results. Six containers: Generator, Store, Search Engine, Visited Set, Profiling Harness, Output. Search Engine components: Frontier, Successor Generator, Explored Set, Goal Test, Node Counter, Path Reconstructor.